In [3]:
```
# Importing required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set visualization style
sns.set_style("whitegrid")

# Display all columns
pd.set_option('display.max_columns', None)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [5]:
```
# Load the dataset
df = pd.read_csv(r"C:\Users\manis\Downloads\Elevate Intern\Task-5\train.csv")

# Display first 5 rows
df.head()
```

Out[5]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S |

In [6]:
```
# Check dataset dimensions
print("Dataset Shape:", df.shape)

# Check column names
print("\nColumn Names:")
print(df.columns.tolist())

# Check data types and non-null values
print("\nDataset Information:")
df.info()
```

```
Dataset Shape: (891, 12)

Column Names:
['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [7]:
```
# Statistical summary of numerical columns
df.describe()
```

Out[7]:

| | PassengerId | Survived | Pclass | Age | SibSp | Parch | Fare |
|---|---|---|---|---|---|---|---|
| **count** | 891.000000 | 891.000000 | 891.000000 | 714.000000 | 891.000000 | 891.000000 | 891.000000 |
| **mean** | 446.000000 | 0.383838 | 2.308642 | 29.699118 | 0.523008 | 0.381594 | 32.204208 |
| **std** | 257.353842 | 0.486592 | 0.836071 | 14.526497 | 1.102743 | 0.806057 | 49.693429 |
| **min** | 1.000000 | 0.000000 | 1.000000 | 0.420000 | 0.000000 | 0.000000 | 0.000000 |
| **25%** | 223.500000 | 0.000000 | 2.000000 | 20.125000 | 0.000000 | 0.000000 | 7.910400 |
| **50%** | 446.000000 | 0.000000 | 3.000000 | 28.000000 | 0.000000 | 0.000000 | 14.454200 |
| **75%** | 668.500000 | 1.000000 | 3.000000 | 38.000000 | 1.000000 | 0.000000 | 31.000000 |
| **max** | 891.000000 | 1.000000 | 3.000000 | 80.000000 | 8.000000 | 6.000000 | 512.329200 |

In [8]:
```
# Survival count
print(df['Survived'].value_counts())

# Survival percentage
print("\nSurvival Percentage:")
print(df['Survived'].value_counts(normalize=True) * 100)
```

```
Survived
0    549
1    342
Name: count, dtype: int64

Survival Percentage:
Survived
0    61.616162
1    38.383838
Name: proportion, dtype: float64
```

In [9]:
```
# Frequency distribution of categorical variables

print("Passenger Class:")
print(df['Pclass'].value_counts())
```

```
print("\nGender Distribution:")
print(df['Sex'].value_counts())

print("\nEmbarkation Port:")
print(df['Embarked'].value_counts())
```

```
Passenger Class:
Pclass
3    491
1    216
2    184
Name: count, dtype: int64

Gender Distribution:
Sex
male      577
female    314
Name: count, dtype: int64

Embarkation Port:
Embarked
S    644
C    168
Q     77
Name: count, dtype: int64
```

In [13]:
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Passenger class
sns.countplot(data=df, x='Pclass', hue='Pclass', ax=axes[0], palette='Set2', legend=False)
axes[0].set_title('Passenger Class Distribution')
axes[0].set_xlabel('Passenger Class')

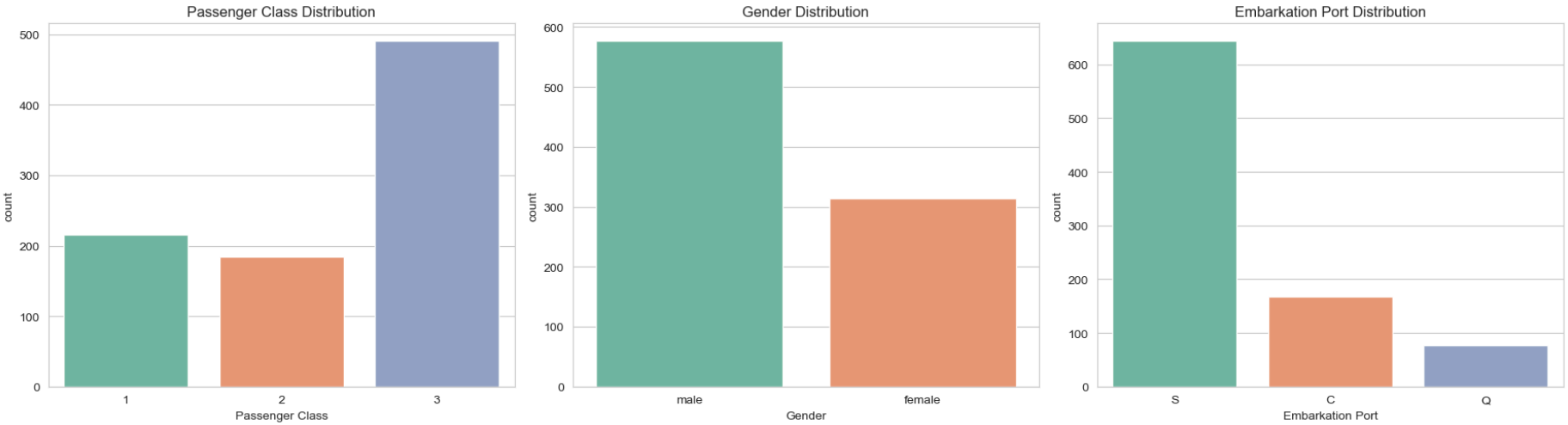
# Gender
sns.countplot(data=df, x='Sex', hue='Sex', ax=axes[1], palette='Set2', legend=False)
axes[1].set_title('Gender Distribution')
axes[1].set_xlabel('Gender')

# Embarkation port
sns.countplot(data=df, x='Embarked', hue='Embarked', ax=axes[2], palette='Set2', legend=False)
axes[2].set_title('Embarkation Port Distribution')
```

```
axes[2].set_xlabel('Embarkation Port')

plt.tight_layout()

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Categorical_Distributions.png",
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```



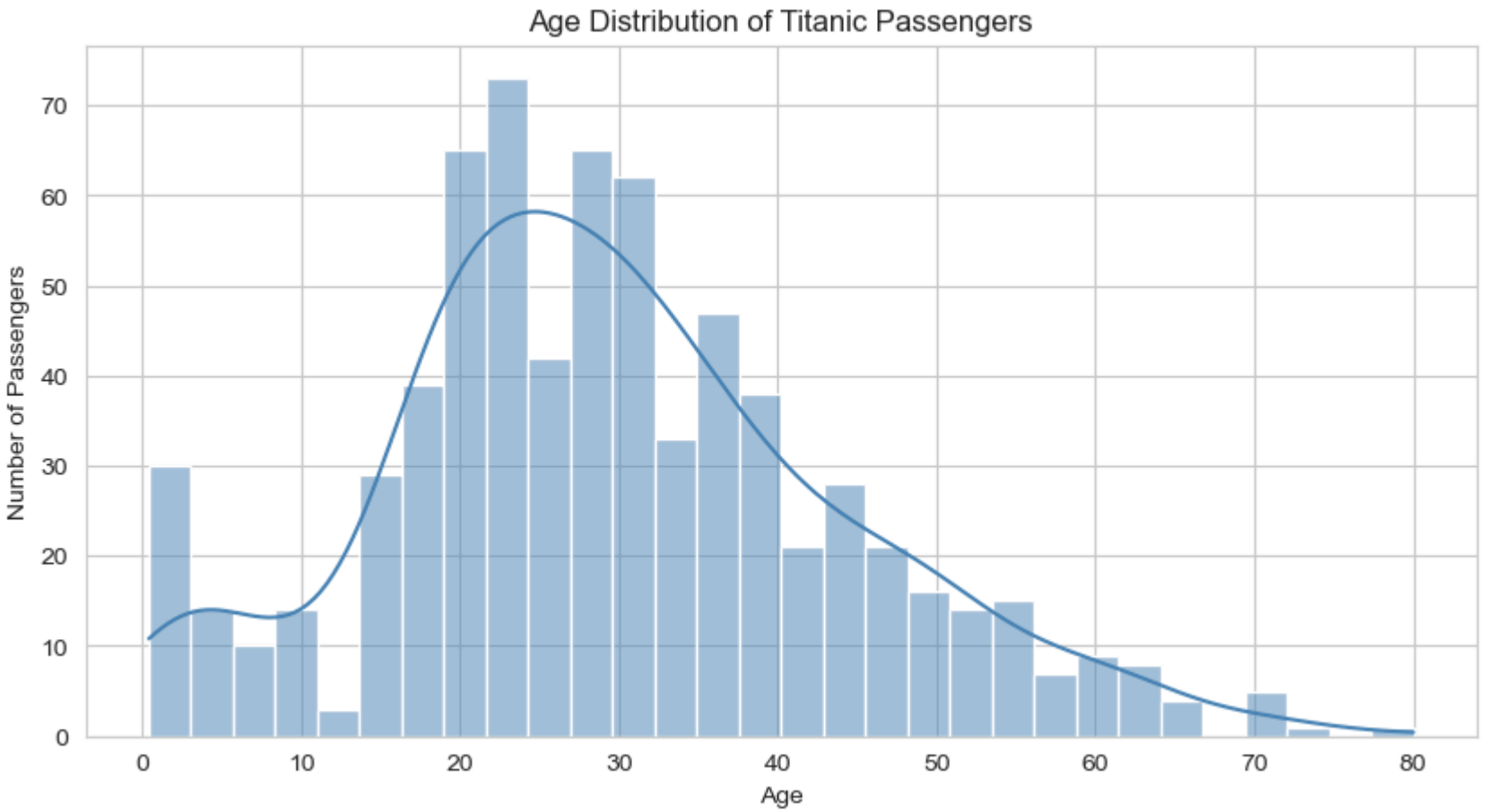

In [15]:
```
plt.figure(figsize=(10, 5))

sns.histplot(data=df, x='Age', bins=30, kde=True, color='steelblue')

plt.title('Age Distribution of Titanic Passengers')
plt.xlabel('Age')
plt.ylabel('Number of Passengers')

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Age_Distribution.png",
    dpi=300,
    bbox_inches='tight'
)
```

```
plt.show()
```




In [16]:
```
plt.figure(figsize=(10, 5))

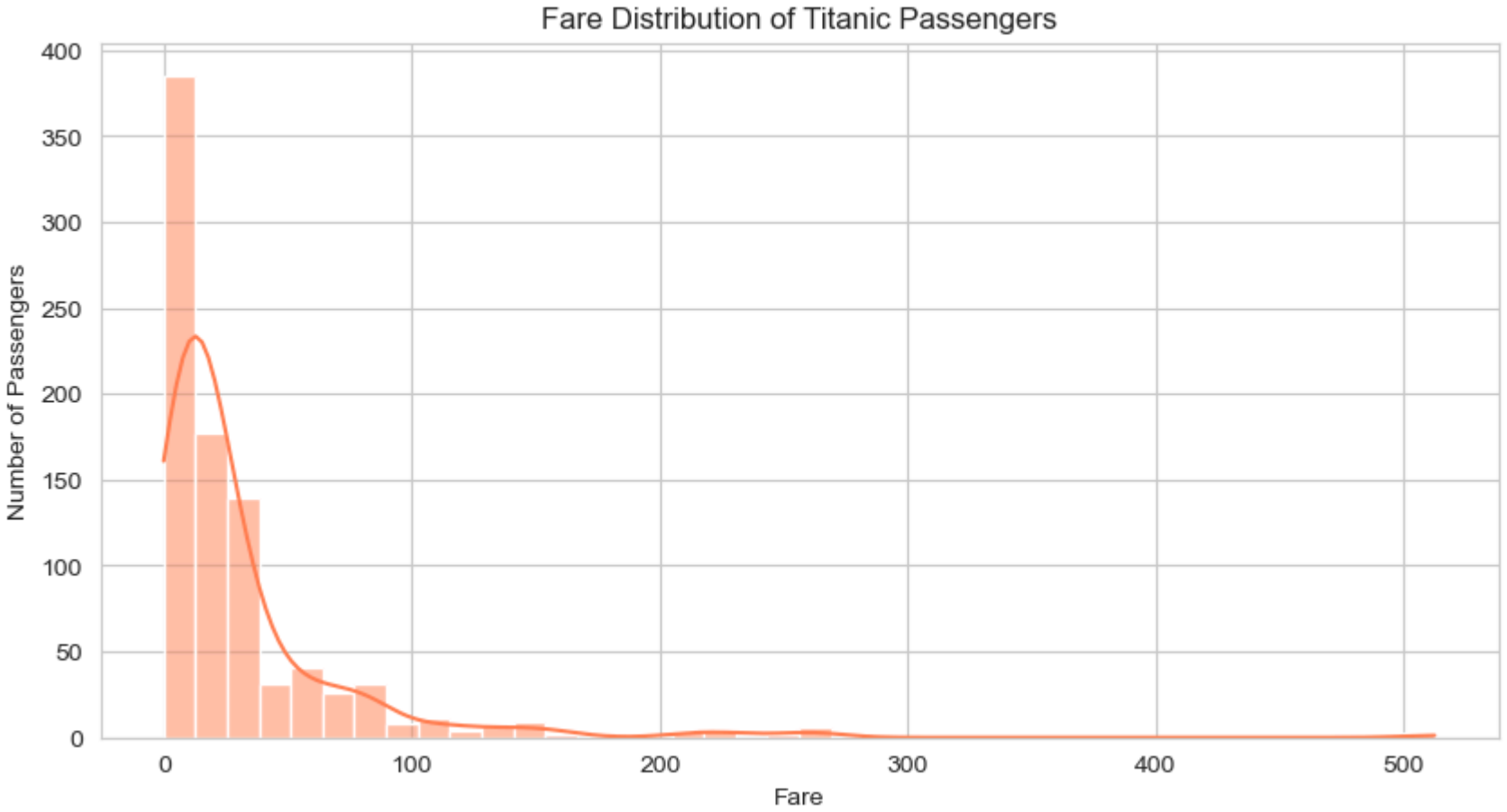
sns.histplot(data=df, x='Fare', bins=40, kde=True, color='coral')

plt.title('Fare Distribution of Titanic Passengers')
plt.xlabel('Fare')
plt.ylabel('Number of Passengers')

plt.savefig(
```

```
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Fare_Distribution.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```




In [ ]:
```
plt.figure(figsize=(10, 4))

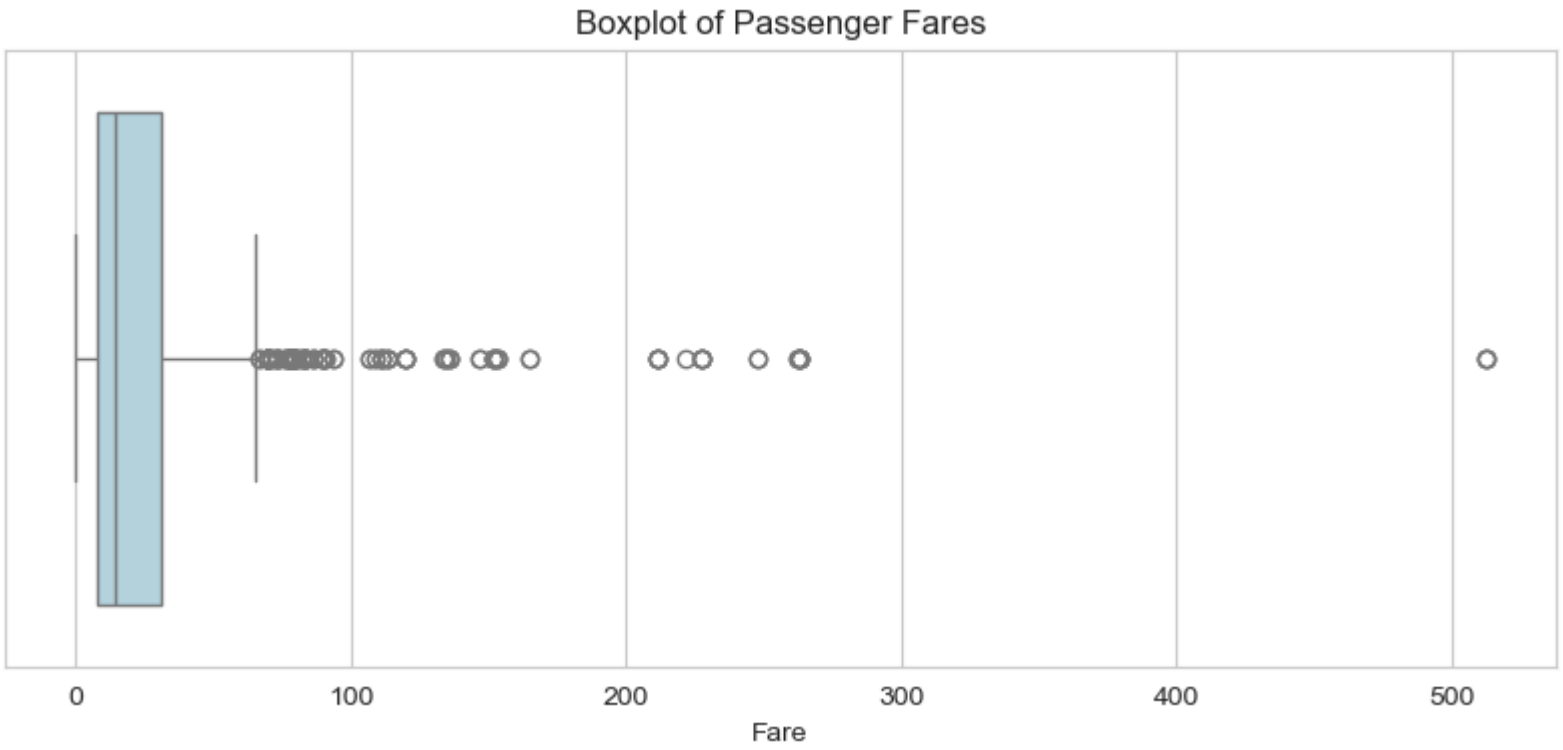
sns.boxplot(x=df['Fare'], color='lightblue')

plt.title('Boxplot of Passenger Fares')
```

```
plt.xlabel('Fare')

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Fare_Boxplot.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```




In [18]:
```
# Calculate Q1 and Q3
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1
```

```
# Calculate outlier boundaries
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
fare_outliers = df[
    (df['Fare'] < lower_bound) |
    (df['Fare'] > upper_bound)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Number of Fare Outliers:", len(fare_outliers))
```

```
Q1: 7.9104
Q3: 31.0
IQR: 23.0896
Lower Bound: -26.724
Upper Bound: 65.6344
Number of Fare Outliers: 116
```

In [19]:

```
# Average fare by passenger class
print(df.groupby('Pclass')['Fare'].agg(['mean', 'median', 'min', 'max']))

# Visualize fare distribution by passenger class
plt.figure(figsize=(10, 5))

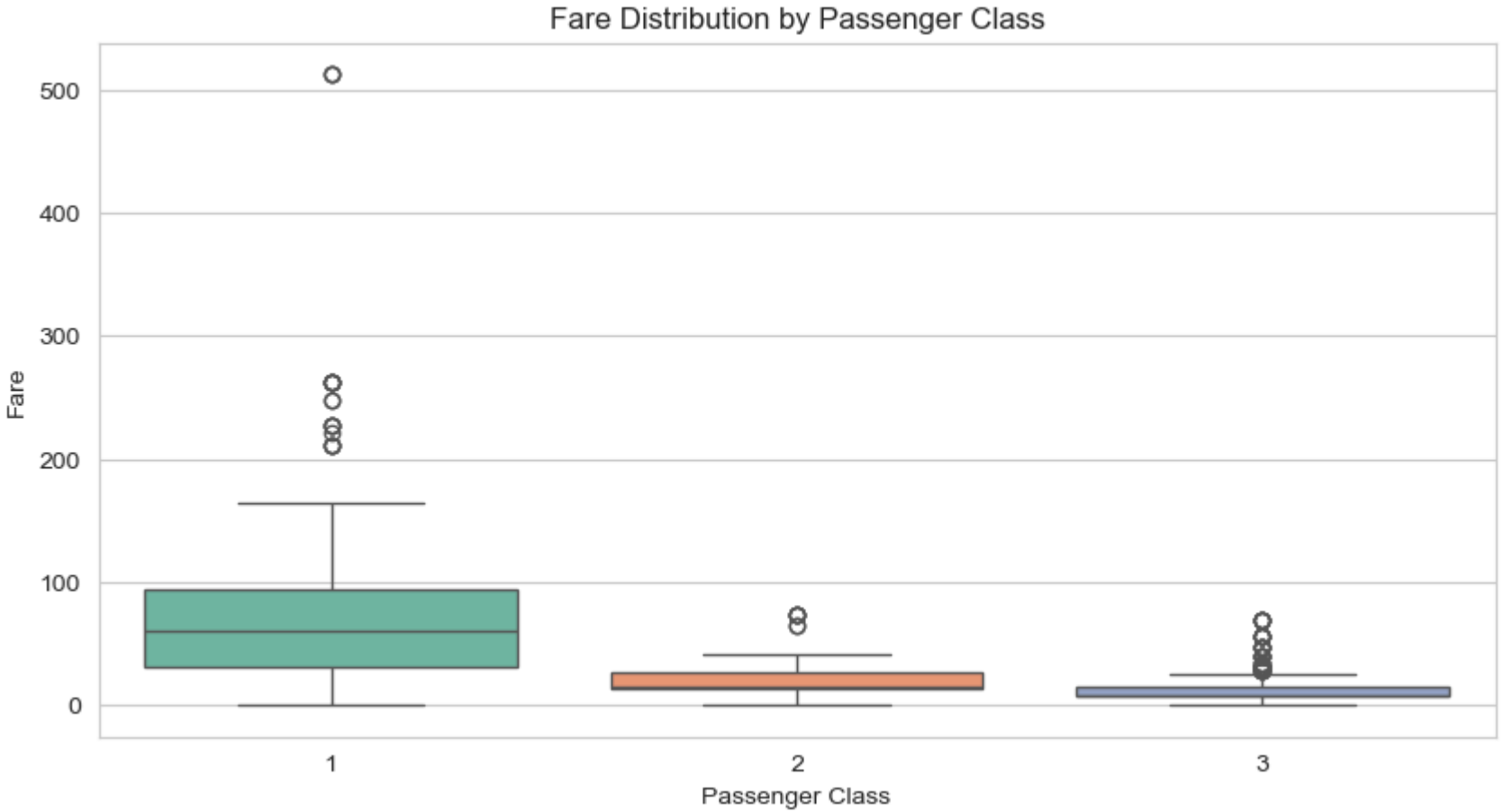
sns.boxplot(data=df, x='Pclass', y='Fare', hue='Pclass', palette='Set2', legend=False)

plt.title('Fare Distribution by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Fare')

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Fare_by_Pclass.png",
    dpi=300,
    bbox_inches='tight'
)
```

```
plt.show()
```

```
            mean    median  min       max
Pclass
1       84.154687  60.2875  0.0  512.3292
2       20.662183  14.2500  0.0   73.5000
3       13.675550   8.0500  0.0   69.5500
```




In [20]:
```
# Survival count by gender
print(pd.crosstab(df['Sex'], df['Survived']))

# Survival percentage by gender
```

```python
gender_survival = pd.crosstab(
    df['Sex'],
    df['Survived'],
    normalize='index'
) * 100

print("\nSurvival Percentage by Gender:")
print(gender_survival)

# Visualization
gender_survival.plot(
    kind='bar',
    stacked=True,
    figsize=(8, 5),
    color=['salmon', 'seagreen']
)

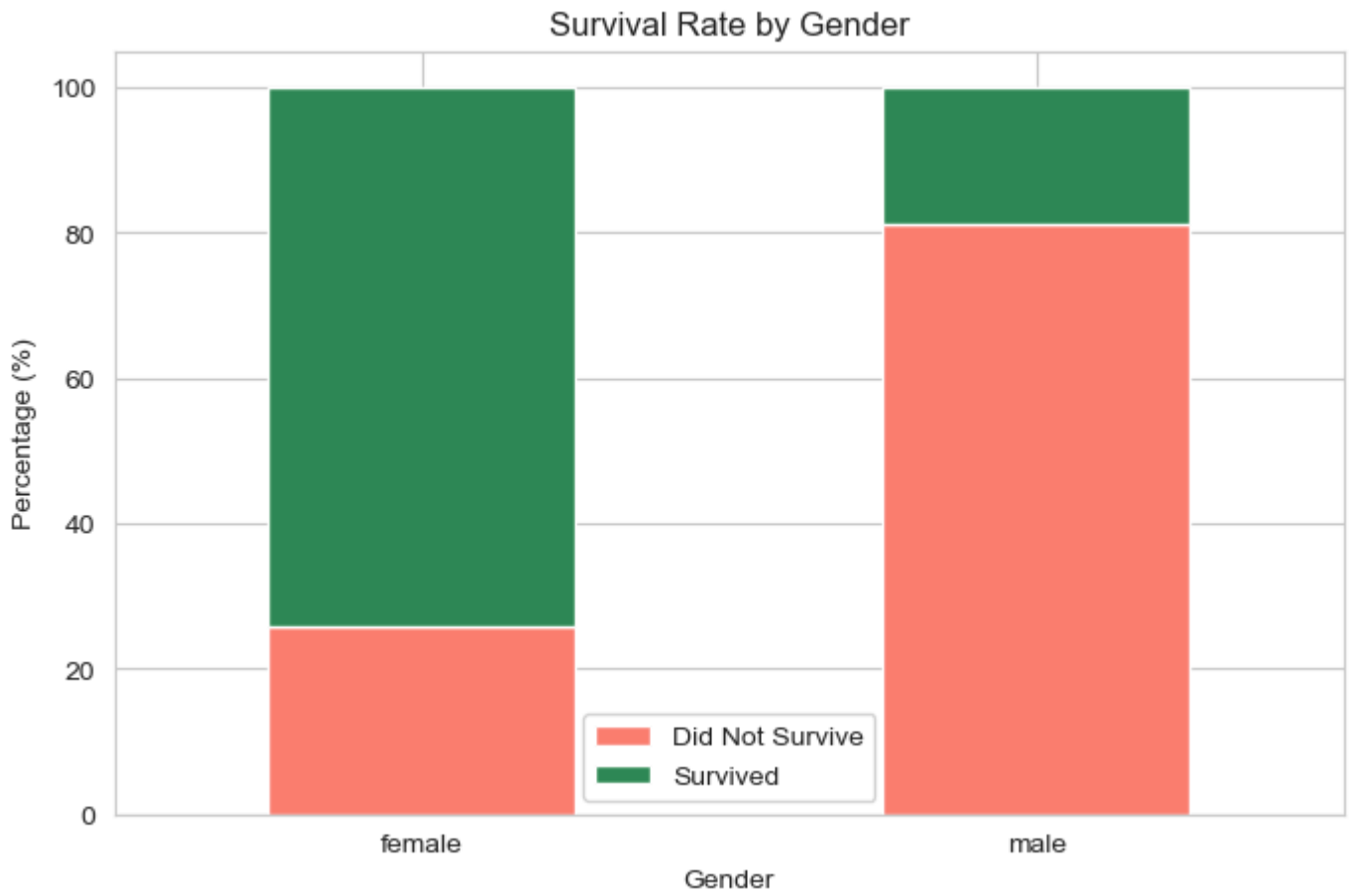
plt.title('Survival Rate by Gender')
plt.xlabel('Gender')
plt.ylabel('Percentage (%)')
plt.xticks(rotation=0)
plt.legend(['Did Not Survive', 'Survived'])

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Survival_by_Gender.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```

```
Survived    0    1
Sex
female     81  233
male      468  109

Survival Percentage by Gender:
Survived          0          1
Sex
female    25.796178  74.203822
male      81.109185  18.890815
```




In [21]:
```
# Survival percentage by passenger class
class_survival = pd.crosstab(
    df['Pclass'],
    df['Survived'],
    normalize='index'
) * 100

print("Survival Percentage by Passenger Class:")
print(class_survival)

# Visualization
class_survival.plot(
```

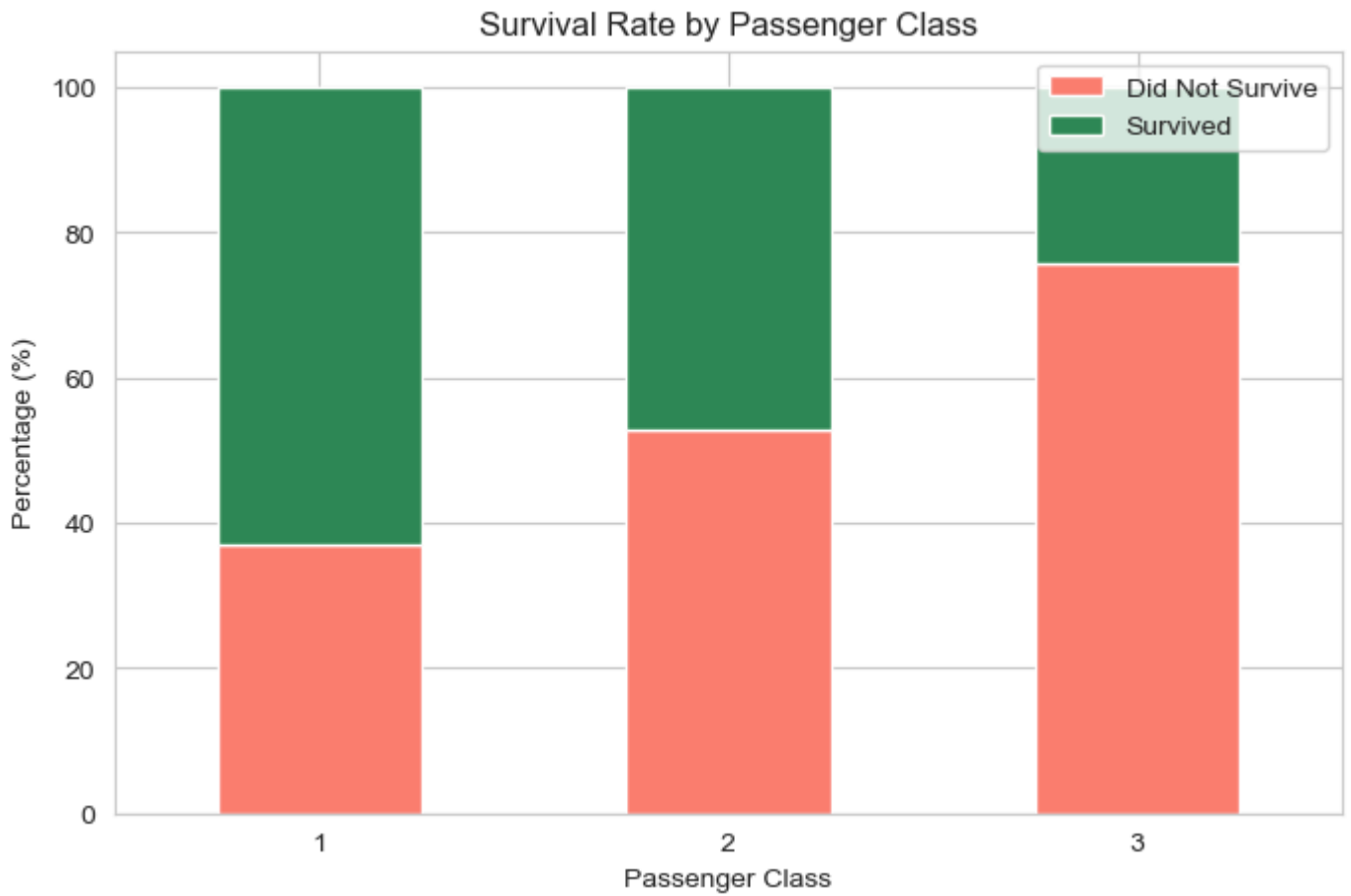
```
    kind='bar',
    stacked=True,
    figsize=(8, 5),
    color=['salmon', 'seagreen']
)

plt.title('Survival Rate by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Percentage (%)')
plt.xticks(rotation=0)
plt.legend(['Did Not Survive', 'Survived'])

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Survival_by_Pclass.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```

```
Survival Percentage by Passenger Class:
Survived          0          1
Pclass
1         37.037037  62.962963
2         52.717391  47.282609
3         75.763747  24.236253
```




In [22]:
```
# Create age groups
df['AgeGroup'] = pd.cut(
    df['Age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['Child', 'Teenager', 'Young Adult', 'Adult', 'Senior']
)

# Survival percentage by age group
age_survival = pd.crosstab(
    df['AgeGroup'],
    df['Survived'],
    normalize='index'
```

```
) * 100

print("Survival Percentage by Age Group:")
print(age_survival)

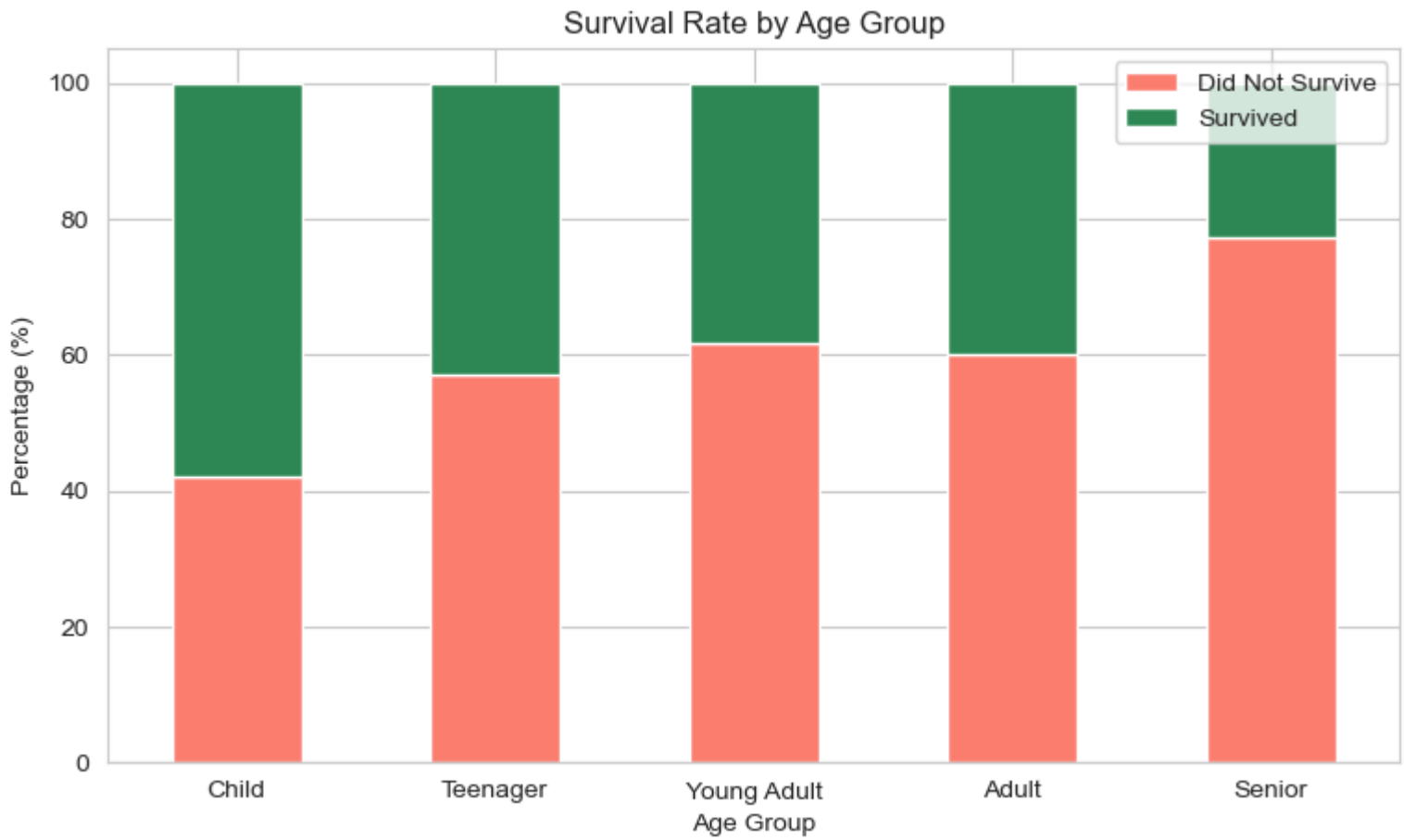
# Visualization
age_survival.plot(
    kind='bar',
    stacked=True,
    figsize=(9, 5),
    color=['salmon', 'seagreen']
)

plt.title('Survival Rate by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Percentage (%)')
plt.xticks(rotation=0)
plt.legend(['Did Not Survive', 'Survived'])

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Survival_by_AgeGroup.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```

```
Survival Percentage by Age Group:
Survived             0          1
AgeGroup
Child        42.028986  57.971014
Teenager     57.142857  42.857143
Young Adult  61.731844  38.268156
Adult        60.000000  40.000000
Senior       77.272727  22.727273
```




In [23]:
```
# Select numerical columns
numeric_df = df.select_dtypes(include=['int64', 'float64'])

# Calculate correlation matrix
correlation = numeric_df.corr()

# Plot heatmap
plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
```

```
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title('Correlation Heatmap of Titanic Dataset')

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Correlation_Heatmap.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```



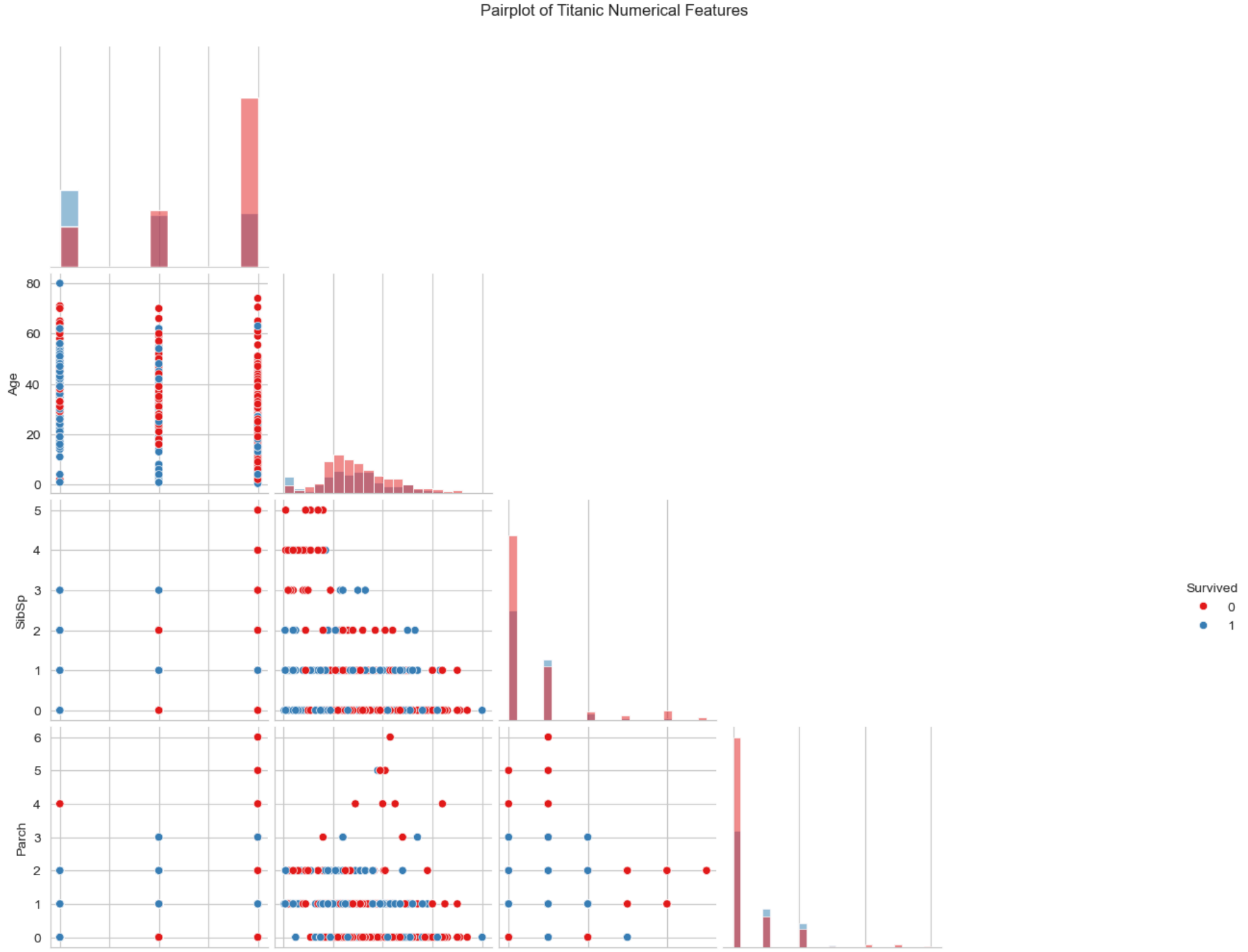

In [24]:
```
# Select important numerical variables
pairplot_data = df[
    ['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
].dropna()
```

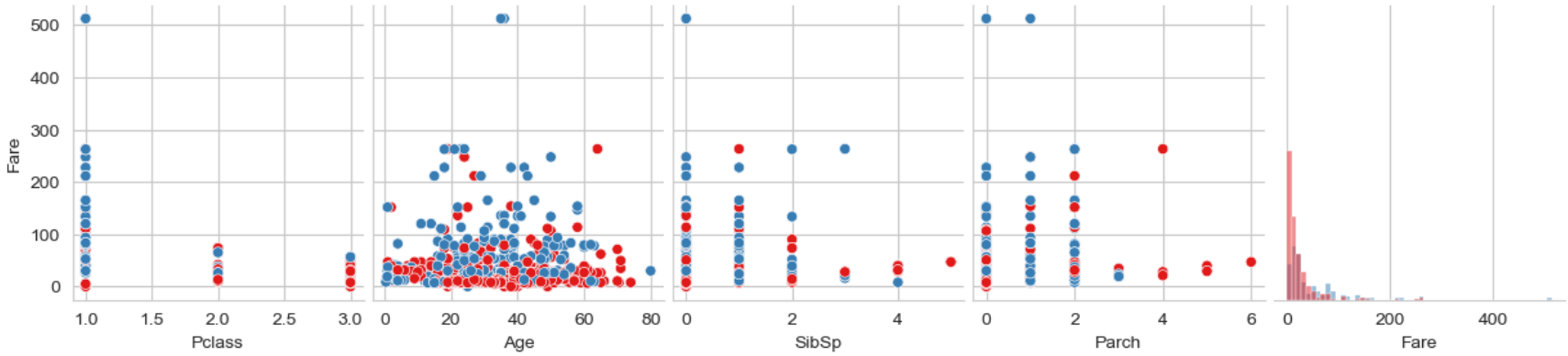
```python
# Generate pairplot
sns.pairplot(
    pairplot_data,
    hue='Survived',
    diag_kind='hist',
    corner=True,
    palette='Set1'
)

plt.suptitle(
    'Pairplot of Titanic Numerical Features',
    y=1.02
)

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Titanic_Pairplot.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```


Pairplot of Titanic Numerical Features
Age
SibSp
Parch
Survived
0
1




In [25]:

```python
# Count missing values
missing_values = df.isnull().sum()

# Calculate missing percentages
missing_percentage = (missing_values / len(df)) * 100

# Create summary table
missing_summary = pd.DataFrame({
    'Missing Values': missing_values,
    'Missing Percentage': missing_percentage
})

missing_summary = missing_summary[missing_summary['Missing Values'] > 0]

print(missing_summary)

# Visualize missing values
plt.figure(figsize=(10, 5))

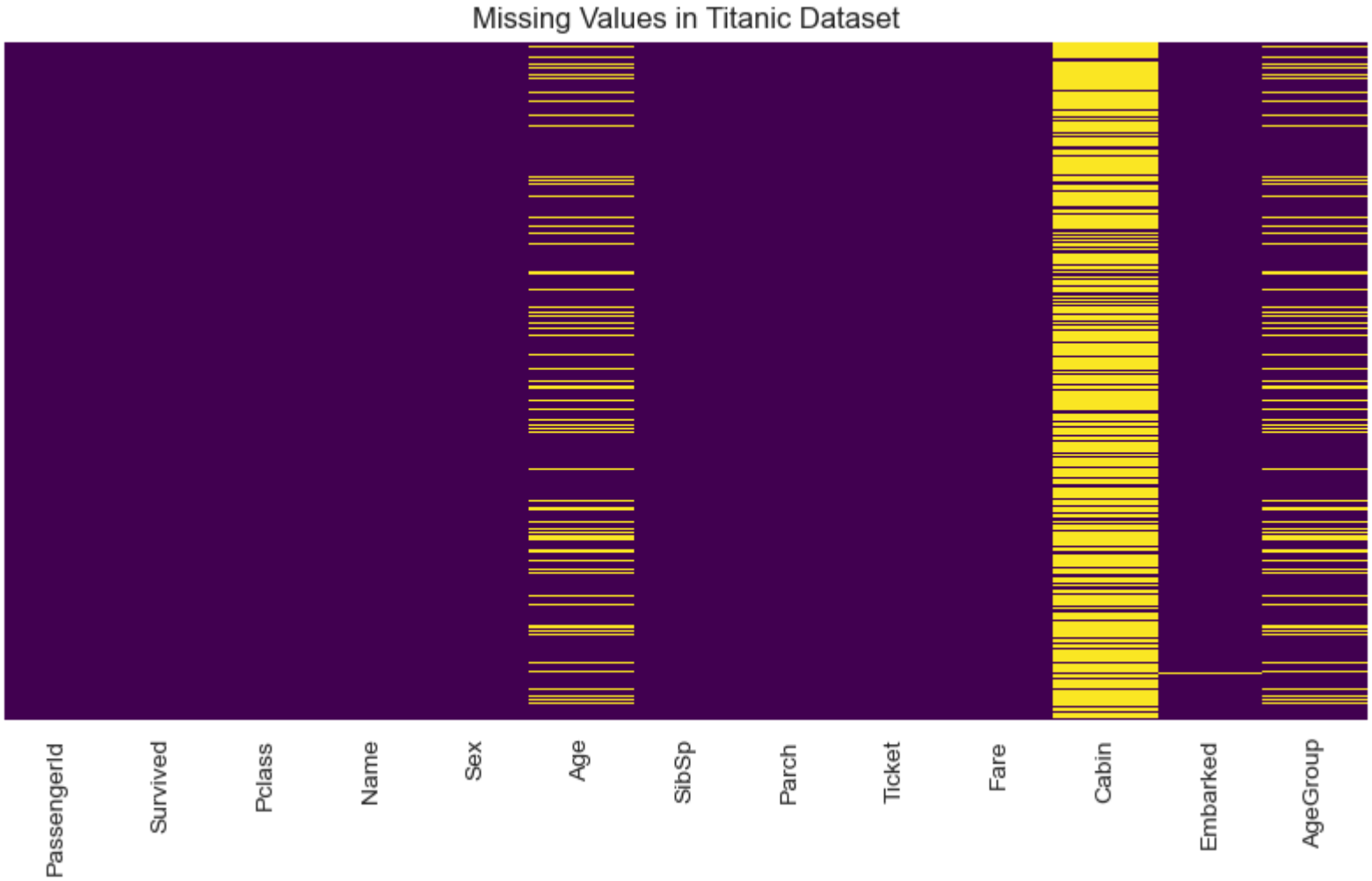
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False)

plt.title('Missing Values in Titanic Dataset')

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Missing_Values_Heatmap.png",
    dpi=300,
    bbox_inches='tight'
)
```

```
plt.show()
```

| | Missing Values | Missing Percentage |
|---|---|---|
| Age | 177 | 19.865320 |
| Cabin | 687 | 77.104377 |
| Embarked | 2 | 0.224467 |
| AgeGroup | 177 | 19.865320 |




In [26]:
```
# Display rows where Embarked is missing
```

```
df[df['Embarked'].isnull()]
```

Out[26]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked | AgeGroup |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **61** | 62 | 1 | 1 | Icard, Miss. Amelie | female | 38.0 | 0 | 0 | 113572 | 80.0 | B28 | NaN | Adult |
| **829** | 830 | 1 | 1 | Stone, Mrs. George Nelson (Martha Evelyn) | female | 62.0 | 0 | 0 | 113572 | 80.0 | B28 | NaN | Senior |

In [27]:
```
# Find most frequent embarkation port
print("Most frequent port:", df['Embarked'].mode()[0])
```

```
Most frequent port: S
```

In [28]:
```
# Create a copy of the original dataset
df_clean = df.copy()

# Fill missing Embarked values with the mode
df_clean['Embarked'] = df_clean['Embarked'].fillna(
    df_clean['Embarked'].mode()[0]
)

# Verify missing values
print("Missing Embarked values:", df_clean['Embarked'].isnull().sum())
```

```
Missing Embarked values: 0
```

In [29]:
```
# Median age by passenger class and gender
age_medians = df_clean.groupby(
    ['Pclass', 'Sex'],
    observed=True
)['Age'].median()

print(age_medians)
```

```
Pclass  Sex
1       female    35.0
        male      40.0
2       female    28.0
        male      30.0
3       female    21.5
        male      25.0
Name: Age, dtype: float64
```

In [30]:
```python
# Fill missing Age using Pclass and Sex group medians
df_clean['Age'] = df_clean.groupby(
    ['Pclass', 'Sex']
)['Age'].transform(
    lambda x: x.fillna(x.median())
)

# Verify missing values
print("Missing Age values:", df_clean['Age'].isnull().sum())

# Check updated dataset
print("\nUpdated Age Summary:")
print(df_clean['Age'].describe())
```

```
Missing Age values: 0

Updated Age Summary:
count    891.000000
mean      29.112424
std       13.304424
min        0.420000
25%       21.500000
50%       26.000000
75%       36.000000
max       80.000000
Name: Age, dtype: float64
```

In [31]:
```python
# Create a cabin availability feature
df_clean['Cabin_Known'] = df_clean['Cabin'].notnull().astype(int)

# Check distribution
print(df_clean['Cabin_Known'].value_counts())
```

```
print("\nCabin Availability Percentage:")
print(df_clean['Cabin_Known'].value_counts(normalize=True) * 100)
```

```
Cabin_Known
0    687
1    204
Name: count, dtype: int64

Cabin Availability Percentage:
Cabin_Known
0    77.104377
1    22.895623
Name: proportion, dtype: float64
```

In [32]:
```
# Survival rate based on cabin information availability
cabin_survival = pd.crosstab(
    df_clean['Cabin_Known'],
    df_clean['Survived'],
    normalize='index'
) * 100

print("Survival Percentage by Cabin Availability:")
print(cabin_survival)

# Visualization
cabin_survival.plot(
    kind='bar',
    stacked=True,
    figsize=(8, 5),
    color=['salmon', 'seagreen']
)

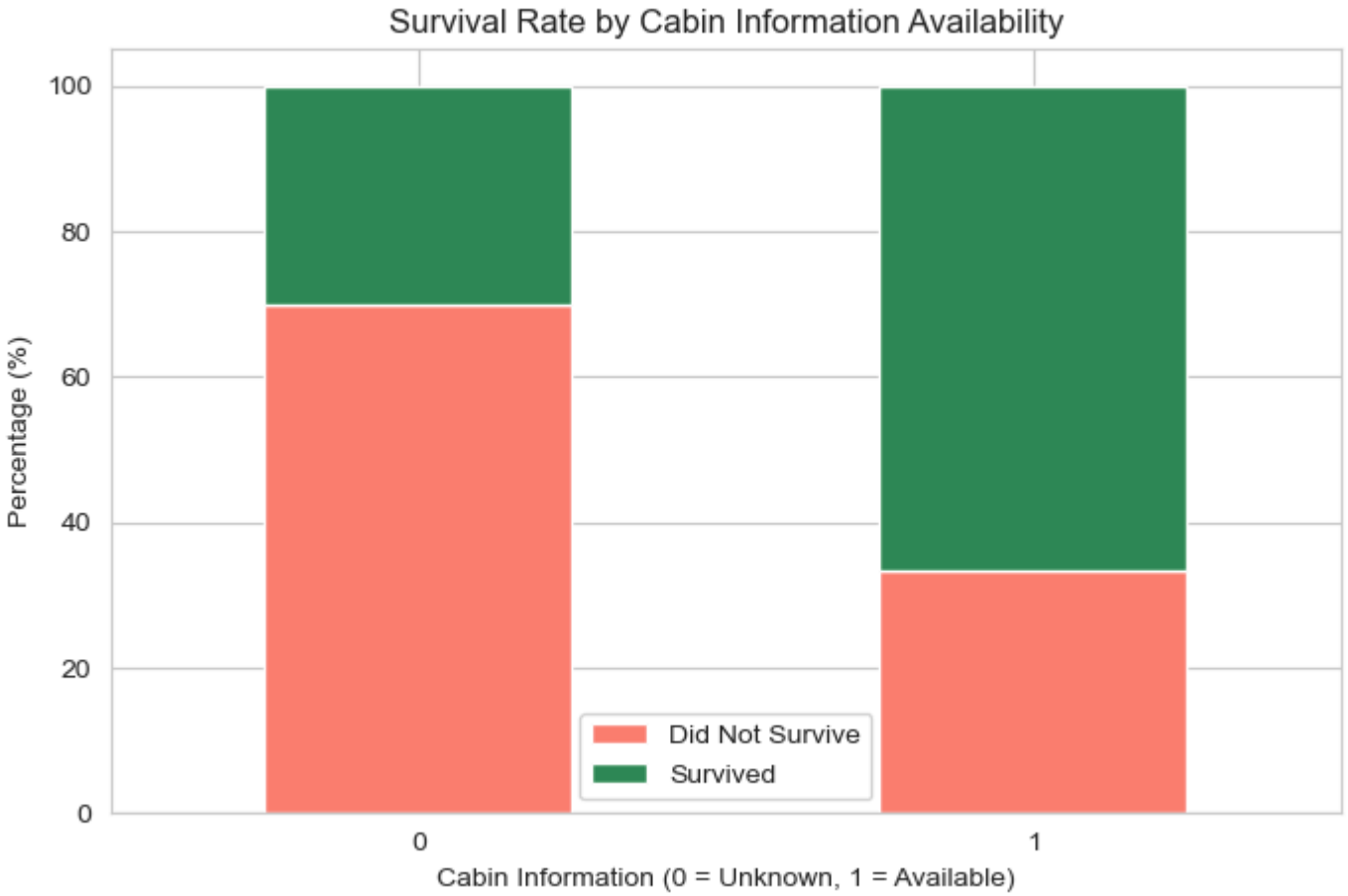
plt.title('Survival Rate by Cabin Information Availability')
plt.xlabel('Cabin Information (0 = Unknown, 1 = Available)')
plt.ylabel('Percentage (%)')
plt.xticks(rotation=0)
plt.legend(['Did Not Survive', 'Survived'])

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Survival_by_Cabin.png",
```

```
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```

```
Survival Percentage by Cabin Availability:
Survived              0          1
Cabin_Known
0             70.014556  29.985444
1             33.333333  66.666667
```



In [33]:
```
# Check missing values after cleaning
print("Missing Values After Cleaning:")
print(df_clean.isnull().sum())

# Check duplicate records
print("\nDuplicate Rows:", df_clean.duplicated().sum())

# Check final dataset shape
print("\nFinal Dataset Shape:", df_clean.shape)

# Display first five rows
df_clean.head()
```

```
Missing Values After Cleaning:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         0
AgeGroup       177
Cabin_Known      0
dtype: int64

Duplicate Rows: 0

Final Dataset Shape: (891, 14)
```

Out[33]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked | AgeGroup | Cabin_Known |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S | Young Adult | 0 |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C | Adult | 1 |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S | Young Adult | 0 |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S | Young Adult | 1 |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S | Young Adult | 0 |

In [34]:

```
# Recreate AgeGroup using cleaned Age values
df_clean['AgeGroup'] = pd.cut(
    df_clean['Age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['Child', 'Teenager', 'Young Adult', 'Adult', 'Senior']
)

# Verify missing values
print("Missing AgeGroup values:", df_clean['AgeGroup'].isnull().sum())

# Final missing-value summary
```

```
print("\nMissing Values:")
print(df_clean.isnull().sum())
```

```
Missing AgeGroup values: 0

Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         0
AgeGroup         0
Cabin_Known      0
dtype: int64
```

In [35]:
```
# Export cleaned dataset
df_clean.to_csv(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\titanic_cleaned.csv",
    index=False
)

print("Cleaned dataset exported successfully!")
```

```
Cleaned dataset exported successfully!
```

In [36]:
```
# Calculate family size
df_clean['FamilySize'] = (
    df_clean['SibSp'] + df_clean['Parch'] + 1
)

# Categorize passengers
df_clean['FamilyType'] = df_clean['FamilySize'].apply(
    lambda x: 'Alone' if x == 1
    else 'Small Family' if x <= 4
```

```
    else 'Large Family'
)

# Survival rate by family type
family_survival = df_clean.groupby(
    'FamilyType',
    observed=True
)['Survived'].mean() * 100

print("Survival Rate by Family Type:")
print(family_survival)
```

```
Survival Rate by Family Type:
FamilyType
Alone           30.353818
Large Family    16.129032
Small Family    57.876712
Name: Survived, dtype: float64
```

In [37]:

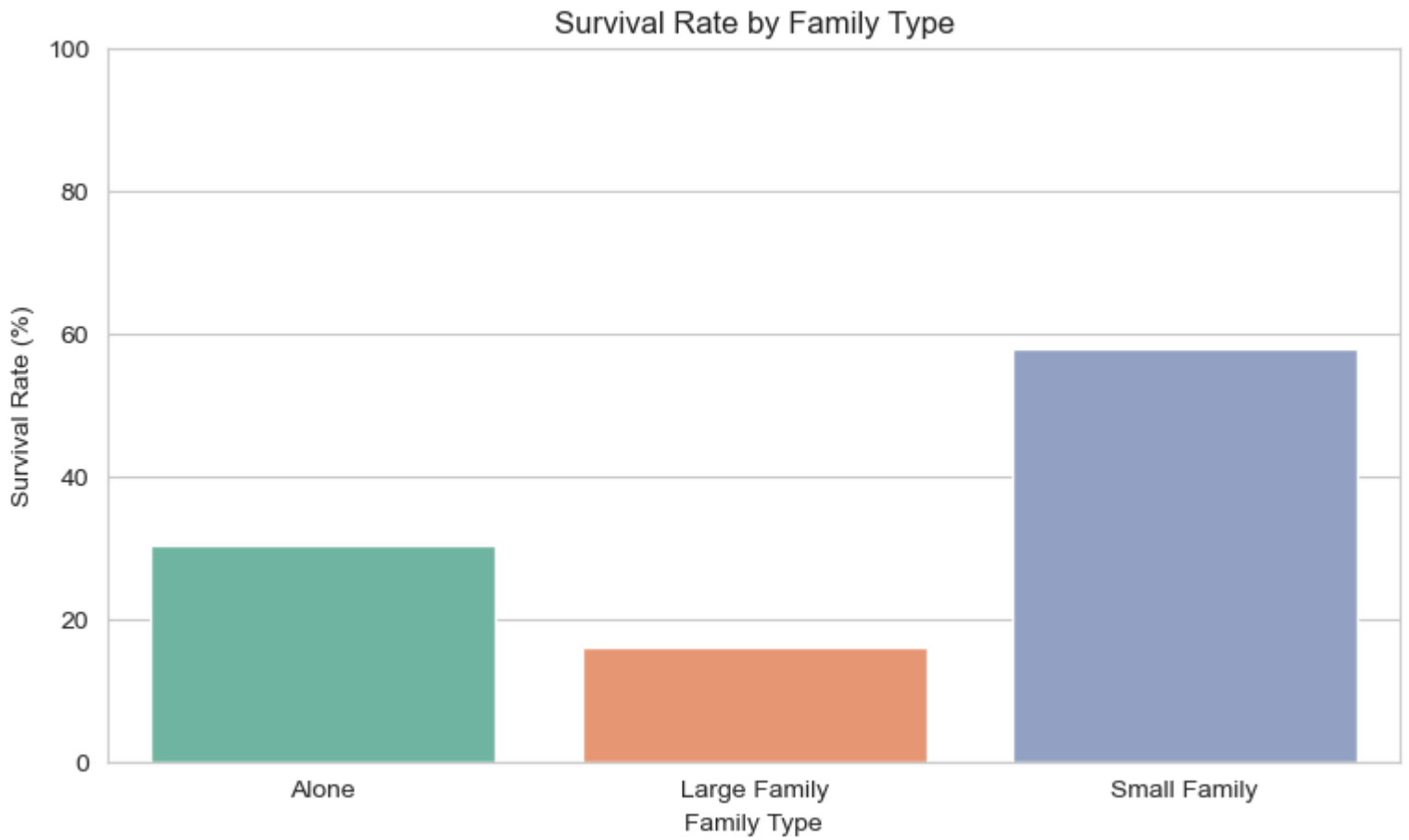
```
plt.figure(figsize=(9, 5))

sns.barplot(
    x=family_survival.index,
    y=family_survival.values,
    hue=family_survival.index,
    palette='Set2',
    legend=False
)

plt.title('Survival Rate by Family Type')
plt.xlabel('Family Type')
plt.ylabel('Survival Rate (%)')
plt.ylim(0, 100)

plt.savefig(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\Survival_by_FamilyType.png",
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```




In [38]:
```
df_clean.to_csv(
    r"C:\Users\manis\Downloads\Elevate Intern\Task-5\titanic_cleaned.csv",
    index=False
)

print("Final cleaned dataset saved successfully!")
print("Final Shape:", df_clean.shape)
```

```
Final cleaned dataset saved successfully!
Final Shape: (891, 16)
```

# Final EDA Findings – Titanic Dataset

## 1. Dataset Overview

- Total passengers analysed: 891
- Total features in the cleaned dataset: 16
- Overall survival rate: 38.38%
- Overall non-survival rate: 61.62%

## 2. Key Findings

- **Gender:** Female passengers had a survival rate of 74.20%, compared with 18.89% for males.
- **Passenger Class:** First-class passengers had the highest survival rate (62.96%), while third-class passengers had the lowest (24.24%).
- **Age:** Children had a survival rate of 57.97%, while senior passengers had a survival rate of 22.73%.
- **Fare:** The fare distribution was highly right-skewed, with 116 potential outliers identified using the IQR method.
- **Cabin Information:** Passengers with recorded cabin information had a survival rate of 66.67%, compared with 29.99% for those without recorded cabin information.
- **Family Size:** Small-family passengers had the highest survival rate (57.88%), while large-family passengers had the lowest (16.13%).

## 3. Data Cleaning

- Filled missing Age values using median imputation grouped by passenger class and gender.
- Filled missing Embarked values using the mode.
- Retained missing Cabin values because approximately 77.10% were unavailable.
- Created Cabin_Known, AgeGroup, FamilySize and FamilyType features.
- No duplicate records were identified.

## 4. Conclusion

Exploratory Data Analysis revealed that gender, passenger class, age, fare and family size were associated with passenger survival outcomes. Female passengers, first-class passengers and passengers travelling with small families showed comparatively higher survival rates.

The analysis demonstrates how statistical summaries, visualizations and feature engineering can help identify meaningful patterns in real-world datasets.